Delta Star: Space-Time A* Path Planning Among Moving Guards

Shubham Patel
Dr. Bhimrao Ambedkar University
Agra
shubhamkumarpatel45@gmail.com

Abstract

A robot on a grid has to reach a goal as fast as it can while guards walk on routes that are known in advance. A normal shortest path search looks only at walls, so it often walks into a guard. This paper describes Delta Star, a C++ planner that adds time to the state. Each state is a triple (x, y, t), and waiting is one of the moves. The guard positions are put in a table before the search starts, so each move check takes constant time. The search is A* with the Manhattan distance as the estimate. On random 30×30 maps, a plain shortest path hit a guard in 16 % of the maps with 2 guards and in 76 % of the maps with 20 guards. Delta Star had no collisions in any map it solved. In the worst-case tests, a 100×100 grid with 1000 time steps and 20 guards needed about 9 million states, 1.75 seconds and 90 MB of memory.

1 Introduction

Path planning means finding a route from a start to a goal. Common methods such as breadth-first search, Dijkstra's algorithm [2] and A* [1, 8] assume that the map does not change while the robot moves. This is not true when other things move on the map. A cell that is free now can be blocked one step later, and sometimes the best plan is just to wait.

This problem shows up in warehouses, where robots share aisles with forklifts and workers that follow a schedule. It also shows up in stealth games with patrolling guards, in drones or rovers that cross an area while scheduled vehicles pass through, and in contest puzzles.

Searching over position and time together is not new. It is often called Space-Time A*, and an early form of it is in the work of Erdmann and Lozano-Pérez [3], and a well-known description for the grid case is given by Silver [4]. This paper presents my own C++ version of that idea. It adds a fast move check, tests, benchmarks, and a separate brute-force checker. The code and a contest-style problem built on it are in the same repository.

The main points of this paper are:

- a planner over (x, y, t) where waiting is a normal move;
- a table of guard positions that makes each move check $O(1)$;
- measurements of time, states and memory in the worst case;
- a comparison with a plain shortest path on random maps, counting collisions.

Preprint.

2 Related work

Planning with time as a dimension. Erdmann and Lozano-Pérez [3] planned paths among moving objects by searching in a space made of position and time. Silver [4] used the same idea on a grid for many agents. In his Cooperative A*, each agent searches over (x, y, time) and avoids the routes that other agents have already reserved. He also gave two faster versions, Hierarchical Cooperative A* and Windowed Hierarchical Cooperative A*, which use a cheaper heuristic or limit how deep the search goes in time. Delta Star uses the same state, but for the simpler case of one robot and guards with fixed routes.

Safe intervals. Safe Interval Path Planning (SIPP) [5] keeps one state for each safe time interval of a cell, and not one state for each time step. The authors show that it keeps the same optimality and completeness as planning with time as a dimension, and that it is much faster in tests with up to 200 moving obstacles. An anytime version [6] returns a first path quickly and improves it while time is left. SIPP-style methods should use fewer states than Delta Star when a cell is blocked only a few times. Delta Star keeps one state for every time step, which is easier to read and to check, but its cost grows with T. This paper does not compare the two, and I leave that for future work.

Many robots. For many robots, Conflict-Based Search [7] finds paths that are complete and optimal. Delta Star plans for one robot only, so it is a much smaller problem.

Search basics. A* [1] uses an estimate that never goes above the true cost to find a best path. Delta Star uses it with the Manhattan distance. Dijkstra's algorithm [2] and breadth-first search are the same idea without an estimate.

3 Problem

We have a grid with walls, a start cell S, a goal cell G, and K guards. The position of guard i at time t is $g_i(t)$, and it is known for every $t \leq T$.

We want the smallest t and a list of cells $v_0 = S, v_1, \ldots, v_t = G$. Each step is a move to a neighbouring cell (up, down, left, right) or a wait. The rules are:

- no cell v_s may hold a guard at time s;
- the robot may not swap places with a guard;
- walls cannot be entered.

4 Method

State. A state is (x, y, t). Because time is part of the state, the same cell at two different times is two different states. This is what lets the robot wait.

Moves. From (x, y, t) the robot can go up, down, left, right, or wait. Each move takes one time step and leads to a state with time $t + 1$.

Move check. A move is allowed if the next cell is not a wall, no guard is on it at time $t + 1$, and the robot is not swapping places with a guard.

Guard table. Before the search, the position of every guard at every time is worked out and stored. A move check then takes $O(1)$ time. My first version checked every guard on every move, which cost $O(K)$ per move.

Estimate. The estimate is the Manhattan distance to the goal (see taxicab geometry [9]). It never goes above the true remaining time, because each step moves at most one cell. So the first time the goal comes out of the priority queue, the time found is the smallest possible.

5 Complexity

Let $W \times H$ be the grid size, T the number of time steps and K the number of guards.

Table 1: Time and memory of each part.

Part	Time	Memory
Build the guard table	$O(T \times K)$	$O(W \times H \times T)$
Search (worst case)	$O(WHT \log(WHT))$	$O(WHT)$
One move check	$O(1)$	–

If the start is in a corner, the robot can only be at cells with $x + y \le t$ at time t. At time t there are $\binom{t+2}{2}$ such cells, so the number of states is at most

$$\sum_{t=0}^{T} \binom{t+2}{2} = \binom{T+3}{3}. \tag{1}$$

For $T = N$ this grows like N^3.

6 Experiments

In the worst-case tests the goal was made unreachable. The search then cannot stop early and has to visit every state it can reach. All numbers below come from my own benchmark program. Times depend on the machine, so other computers will give different values.

6.1 Time and states

Table 2 shows square grids with $T = N$. Every measured count is just below the bound $\binom{T+3}{3}$ from Equation (1), which is 1 771, 39 711, 176 851 and 302 621 for these four rows. Figure 1(a) plots time against N on log axes, with a dashed line for N^3. The measured line follows the dashed line closely, except that the point at $N = 100$ lies a little above it.

Table 2: Square grids with $T = N$ and an unreachable goal.

Grid $N \times N$	T	States expanded	Time
20×20	20	1 746	0.2 ms
60×60	60	39 558	5.5 ms
100×100	100	176 493	39 ms
120×120	120	301 927	53 ms

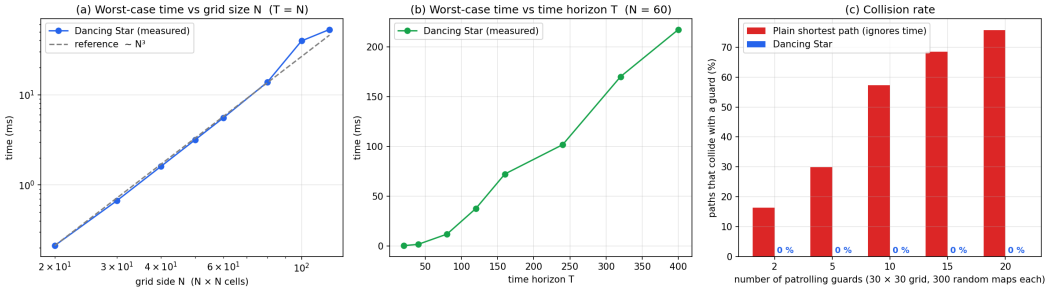


Figure 1: Benchmark results. (a) Worst-case time against grid size N with $T = N$, on log axes; the dashed line is N^3. (b) Worst-case time against the time limit T on a 60×60 grid. (c) Share of paths that collide with a guard on 30×30 random maps, 300 maps for each number of guards.

Figure 1(b) keeps the grid fixed at 60×60 and changes T from 20 to 400. The time grows with T and reaches about 0.2 seconds at $T = 400$.

6.2 Larger grids with 20 guards

Table 3 shows bigger runs. Going from $T = 500$ to $T = 1000$ on the 100×100 grid took the time from 0.81 s to 1.75 s. Memory follows about $9 \times W \times H \times (T + 1)$ bytes. For 100×100 and $T = 1000$ this is about 90 MB, the same as the measured value.

Table 3: Worst cases with 20 guards and an unreachable goal.

Grid	T	States expanded	Time	Peak memory
50×50	200	375 748	0.05 s	about 11 MB
100×100	500	4 009 148	0.81 s	about 47 MB
100×100	1000	8 997 648	1.75 s	about 90 MB

6.3 Collisions

We compared Delta Star with a plain shortest path that looks only at walls and ignores time. Both ran on random 30×30 maps with 2, 5, 10, 15 and 20 guards, using 300 maps for each guard count (Figure 1(c)). The plain shortest path hit a guard in 16 % of the maps with 2 guards and in 76 % of the maps with 20 guards. Reading the bars in the figure, the values for 5, 10 and 15 guards are about 30 %, 57 % and 68 %. Delta Star had 0 collisions at every guard count. It solved about 1 380 of the 1 399 maps. In the others, no safe path existed within the time limit.

6.4 An example run

Figure 2 shows one small map with walls and 2 guards. The robot goes from S in the top left corner to G in the bottom right corner in 16 steps. The Manhattan distance between S and G is $10 + 6 = 16$, so the robot did not lose any time.

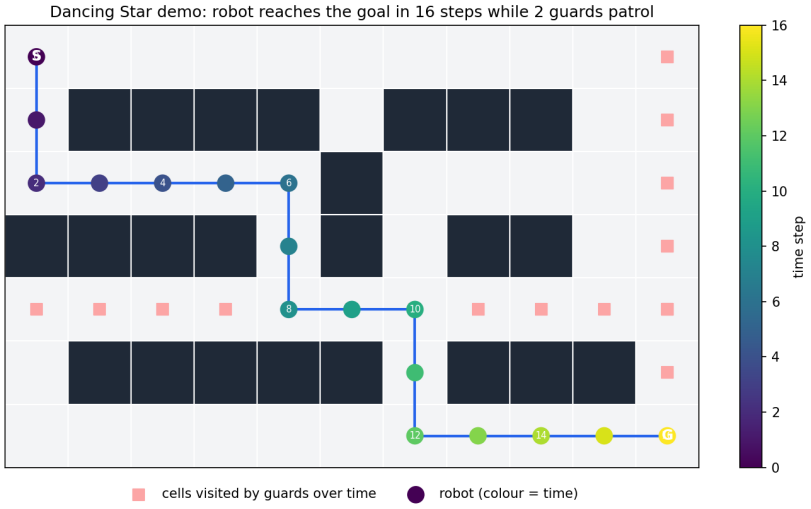


Figure 2: A demo path. Dark cells are walls, pink squares are cells that guards visit over time, and the robot's colour shows the time step.

6.5 Checking the answers

A separate brute-force solver, written by me in Python, was run against the C++ code on 300 random small maps. The two never disagreed.

7 Limits

- Time moves in whole steps. A move and a wait both cost 1.
- Only four directions are allowed. Diagonal moves would break the Manhattan estimate.
- Guard routes must be fixed and known. If guards move in ways we cannot predict, the planner has to run again every few steps.
- Guards do not react to the robot. Two guards may stand on the same cell.
- The start cell must be free at $t = 0$.
- The code allows at most 32 guards, because it uses one bit per guard in a `uint32_t`.
- "Not found" means the goal cannot be reached within T steps. It does not mean it can never be reached.
- Memory grows with $W \times H \times T$, so long time limits on big grids need a lot of memory.

8 Conclusion

Adding time to the state lets A* handle guards that move on known routes, and it lets the robot wait when waiting is best. A table of guard positions makes each move check take constant time. In these tests the planner never hit a guard, while a plain shortest path hit one in up to 76 % of the maps. The price is time and memory that grow with the number of time steps.

Possible next steps are to compare Delta Star with SIPP on the same maps, to make waiting cost more, to add a penalty for cells next to a guard, to let guards move at random and plan again every few steps, and to plan for several robots together.

Code availability

The code, tests, benchmarks and the contest-style problem are in the Delta Star repository (MIT license).

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[2] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.

[3] M. Erdmann and T. Lozano-Pérez. On multiple moving objects. *Algorithmica*, 2:477–521, 1987.

[4] D. Silver. Cooperative pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, 1(1):117–122, 2005.

[5] M. Phillips and M. Likhachev. SIPP: Safe interval path planning for dynamic environments. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 5628–5635, 2011.

[6] V. Narayanan, M. Phillips, and M. Likhachev. Anytime safe interval path planning for dynamic environments. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4708–4715, 2012.

[7] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.

[8] Wikipedia contributors. A* search algorithm. *Wikipedia, The Free Encyclopedia*. `https://en.wikipedia.org/wiki/A*_search_algorithm`

[9] Wikipedia contributors. Taxicab geometry. *Wikipedia, The Free Encyclopedia*. `https://en.wikipedia.org/wiki/Taxicab_geometry`